MIT-ADT UNIVERSITY
PUNE, INDIA
A Leap towards World Class Education

# CONFIGURABLE PAYMENT PROCESSING SYSTEM

Using the **Strategy Design Pattern**


SUBMITTED BY

## Om Yogesh Thombre

**ROLL NO. 31**


### PROBLEM STATEMENT

To develop a configurable payment processing system using the Strategy Design Pattern.

| | | | |
|---|---|---|---|
| **CARD** Payment | **UPI** Payment | **WALLET** Payment | **NET BANKING** Payment |

01 • PROJECT OVERVIEW

# Configurable Payment Processing System

A modular payment-processing application designed around the Strategy Design Pattern.

| DESIGN PATTERN | CORE CONTEXT |
| --- | --- |
| Strategy Pattern | PaymentProcessor |
| **STRATEGIES** | **LANGUAGE** |
| Card • UPI • Wallet • Net Banking | Python |

## WHY STRATEGY PATTERN?

The Strategy Design Pattern allows different payment algorithms to be encapsulated as independent strategies. The PaymentProcessor works with the common PaymentStrategy interface, so the selected payment method can change without changing the processor itself.

## SYSTEM COMPONENTS

| | |
| --- | --- |
| **PaymentStrategy** | Common interface for all payment methods. |
| **Concrete Strategies** | Card, UPI, Wallet and Net Banking implementations. |
| **PaymentProcessor** | Context class that delegates processing to the selected strategy. |

02 • PROGRAM CODE

# Python Implementation

The following program uses an abstract PaymentStrategy and concrete strategies for different payment methods.

```
from abc import ABC, abstractmethod
from datetime import datetime

class PaymentStrategy(ABC):
@abstractmethod
def process_payment(self, amount):
pass

class CardPaymentStrategy(PaymentStrategy):
def process_payment(self, amount):
print(f"[CARD] Processing payment of Rs. {amount:.2f}")
return True

class UPIPaymentStrategy(PaymentStrategy):
def process_payment(self, amount):
print(f"[UPI] Processing payment of Rs. {amount:.2f}")
return True

class WalletPaymentStrategy(PaymentStrategy):
def process_payment(self, amount):
print(f"[WALLET] Processing payment of Rs. {amount:.2f}")
return True

class NetBankingStrategy(PaymentStrategy):
def process_payment(self, amount):
print(f"[NET BANKING] Processing payment of Rs. {amount:.2f}")
return True

class PaymentProcessor:
def __init__(self, strategy):
self.strategy = strategy

def set_strategy(self, strategy):
self.strategy = strategy
```

**Key idea:** PaymentProcessor delegates the payment operation to the selected strategy.

# Python Implementation

The following program uses an abstract PaymentStrategy and concrete strategies for different payment methods.

```
def process_payment(self, amount):
return self.strategy.process_payment(amount)

def main():
strategies = {
"1": CardPaymentStrategy(),
"2": UPIPaymentStrategy(),
"3": WalletPaymentStrategy(),
"4": NetBankingStrategy()
}

print("=" * 55)
print(" CONFIGURABLE PAYMENT PROCESSING SYSTEM")
print("=" * 55)
print("1. Card Payment")
print("2. UPI Payment")
print("3. Wallet Payment")
print("4. Net Banking")
print("-" * 55)

choice = input("Enter payment method: ")
try:
amount = float(input("Enter amount: "))
except ValueError:
print("Invalid amount.")
return

if choice not in strategies or amount <= 0:
print("Invalid input.")
return

processor = PaymentProcessor(strategies[choice])
success = processor.process_payment(amount)
```

# Payment Processing Algorithm

Step-by-step execution from user input to transaction completion.

| | | |
|---|---|---|
| 01 | **START** | Initialize the payment processing system. |
| 02 | **DISPLAY** | Display Card, UPI, Wallet and Net Banking options. |
| 03 | **INPUT** | Read the selected payment method and amount. |
| 04 | **VALIDATE** | Verify that the method is valid and amount is greater than zero. |
| 05 | **SELECT** | Select the corresponding PaymentStrategy object. |
| 06 | **CONFIGURE** | Create PaymentProcessor with the selected strategy. |
| 07 | **PROCESS** | Call process_payment(amount). |
| 08 | **CHECK** | Check whether payment processing is successful. |
| 09 | **COMPLETE** | Display SUCCESS and transaction ID, or display FAILED. |
| 10 | **END** | Terminate the program. |

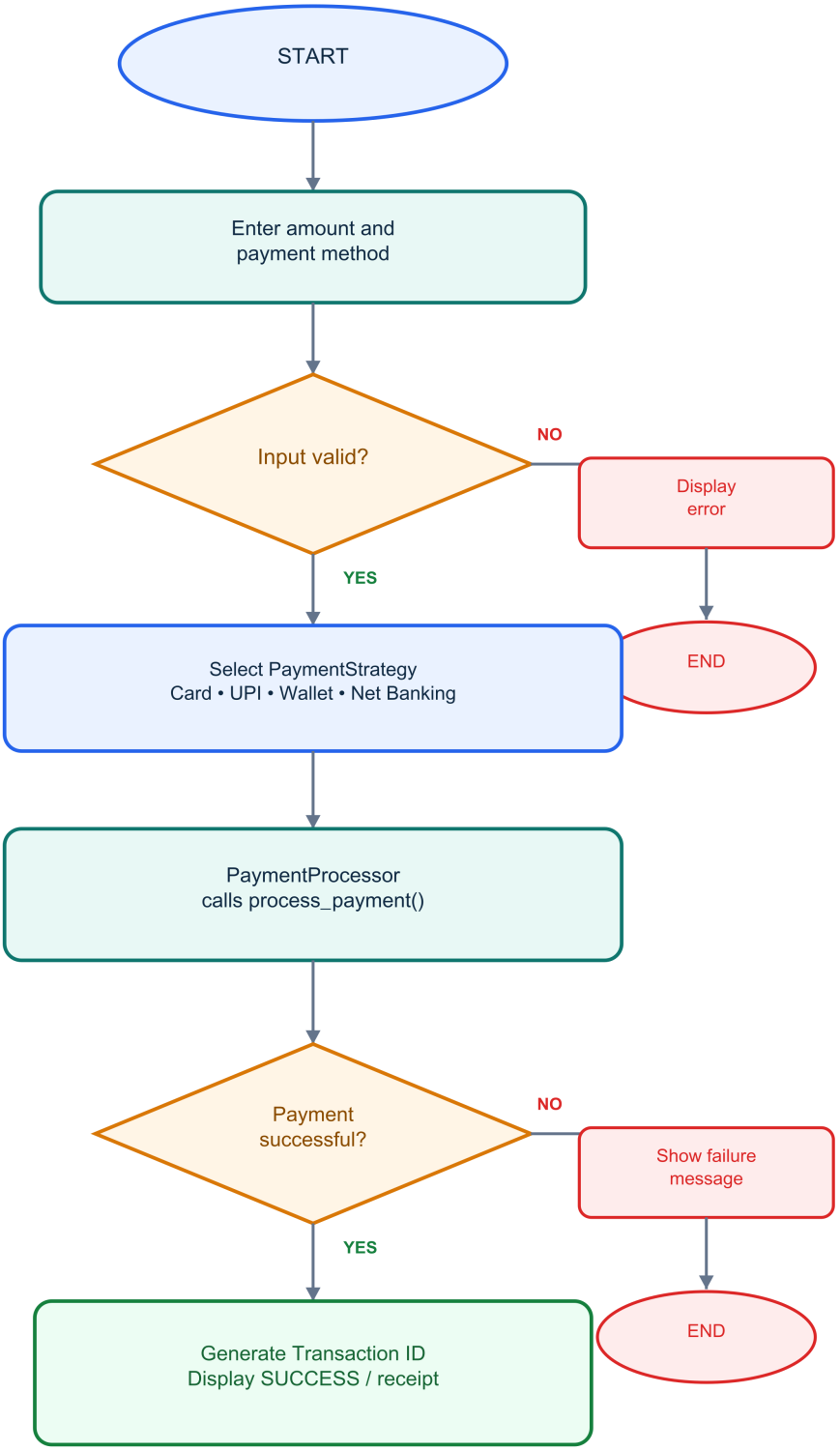
## PSEUDOCODE

```
START Display payment methods Read choice and amount Validate input IF invalid THEN Display error STOP END IF Select
PaymentStrategy Create PaymentProcessor(strategy) Call process_payment(amount) IF successful THEN Generate transaction ID
Display SUCCESS ELSE Display FAILED END IF STOP
```

# Flowchart

The flow uses standard Start/End, Process and Decision symbols to represent the program logic.

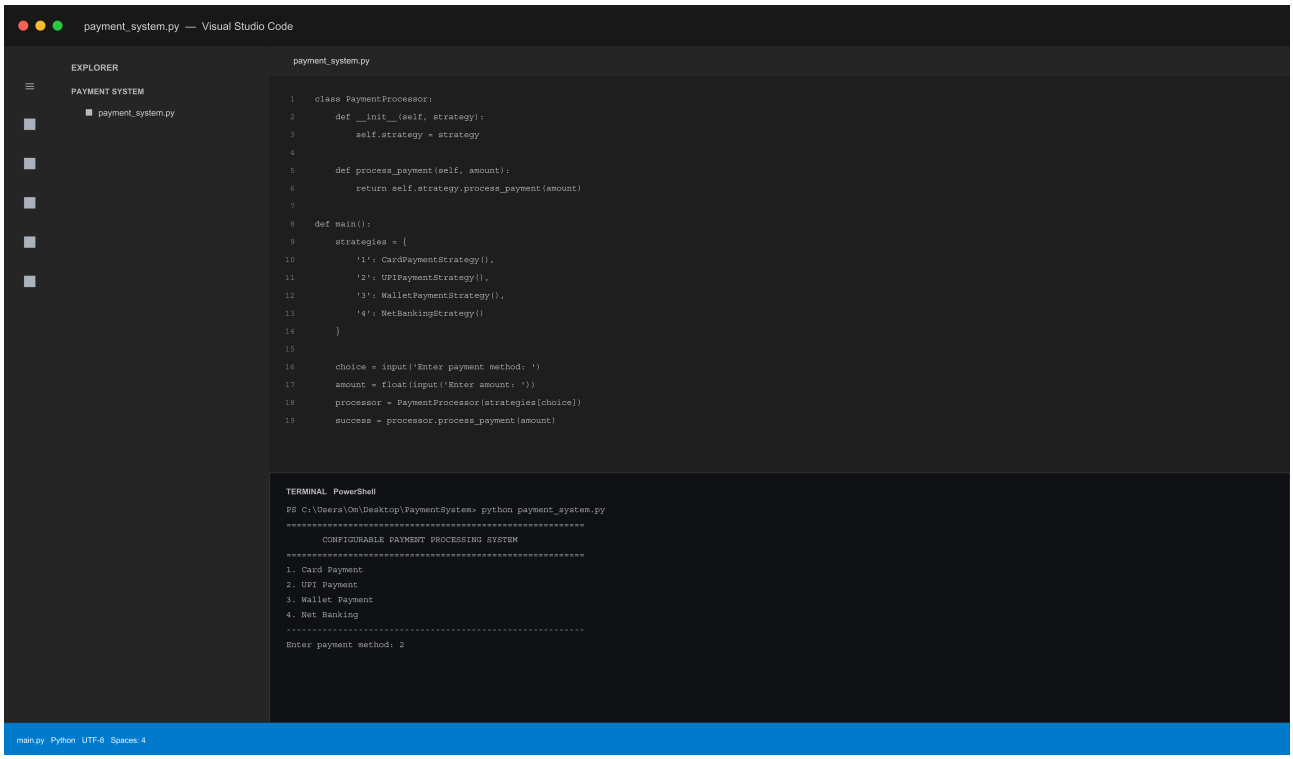
| OVAL | RECTANGLE | DIAMOND |
|---|---|---|
| Start / End | Process / Input | Decision |



05 • OUTPUT SCREENSHOT 01

# Program Launch

The application starts in the VS Code integrated terminal and displays the available payment methods.

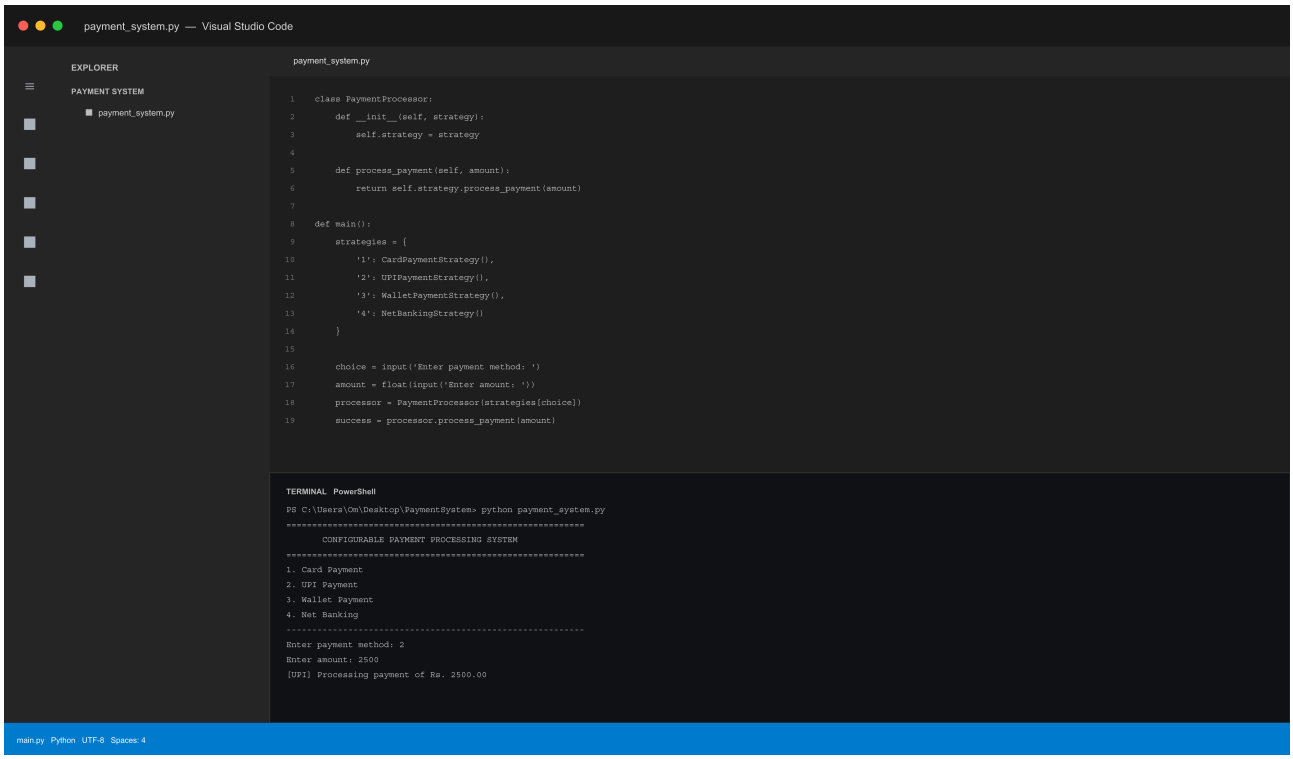



**Execution stage:** Program is running and waiting for the user's payment-method selection.

05 • OUTPUT SCREENSHOT 02

# Payment Input & Strategy Execution

The user selects UPI and enters the transaction amount. The UPI strategy processes the payment.

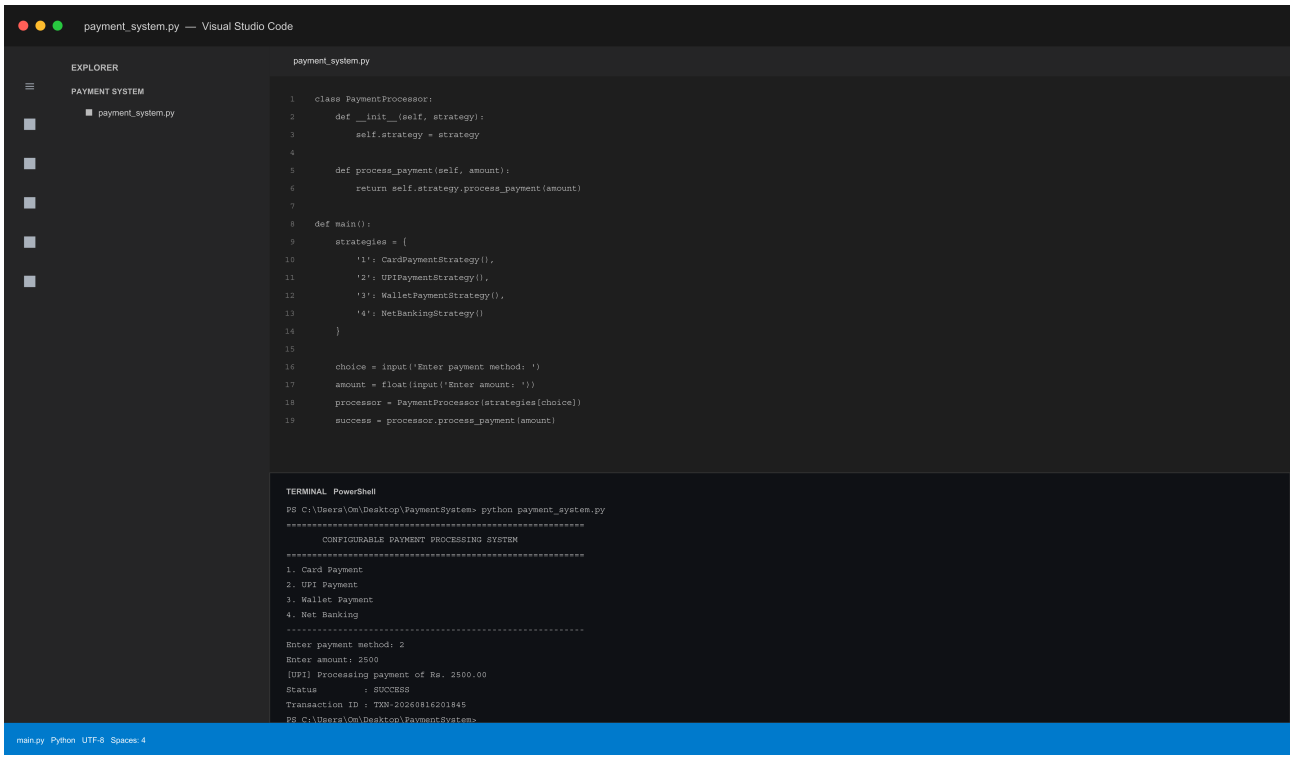



**Execution stage:** The selected UPI strategy is invoked through PaymentProcessor.

05 • OUTPUT SCREENSHOT 03

# Successful Transaction

The system reports SUCCESS and displays a generated transaction ID.




**Execution stage:** The transaction completes successfully and a transaction ID is generated.

# Project Result

The configurable payment processing system was successfully implemented using the Strategy Design Pattern. The program supports multiple payment strategies and allows the required strategy to be selected dynamically.

## ADVANTAGES OF THE DESIGN

| | |
|---|---|
| **FLEXIBLE** | Payment methods can be switched without changing the core processor. |
| **MODULAR** | Each payment method has its own independent strategy class. |
| **MAINTAINABLE** | Payment logic is separated from the context that uses it. |
| **EXTENSIBLE** | New payment strategies can be added with minimal modification. |

## CONCLUSION

The Strategy Design Pattern provides a clean and scalable approach for configurable payment processing. By separating payment algorithms into individual strategy classes, the system remains easy to understand, test, maintain and extend.

**Om Yogesh Thombre • Roll No. 31**
MIT-ADT University, Pune, India